

Dossier de Projet Professionnel

Titre professionnel : Concepteur Développeur d'Applications (CDA)



TaskFlow

Application de gestion de projets

Réalisé par : Pélagie AINTANGAR

Formation : Développement Web

Centre de formation : La Plateforme

Session juillet 2026

SOMMAIRE

Introduction

- Présentation du projet
- Objectifs
- Compétences couvertes

1. Contexte et Analyse des besoins

1.1 Contexte du projet

1.2 Objectif

1.3 Problématique

1.4 Public cible

1.5 Besoins fonctionnels

1.6 Contraintes

2. Gestion de projet

2.1 Méthodologie Agile

2.2 Organisation et suivi du projet avec Trello

2.3 Gestion Git/GitHub

2.4 Planification des tâches

3. Maquettage et conception

3.1 Wireframes

3.2 Maquettes Figma

3.3 User Flow

3.4 Diagramme cas d'utilisation UML

3.5 Diagramme de séquence UML

3.6 Diagramme de classes UML

4. Architecture logicielle

4.1 Architecture globale

4.2 Architecture multicouche

4.3 Choix architecture monolithique

4.4 Alternatives étudiées

- microservices
- hexagonale

4.5 Stratégie sécurité

- JWT
- bcrypt
- middleware
- validation
 - Isolation des services avec Docker
 - Reverse proxy Traefik
 - Sauvegarde et restauration de la base de données

5. Base de données

5.1 Choix PostgreSQL

5.2 Modèle Conceptuel de Données - MCD

5.3 Modèle Logique de Données - MLD

5.4 Relations

5.5 Intégrité référentielle

5.6 Sauvegarde et restauration de la base de données

6. Développement backend

6.1 Organisation du projet

6.2 Routes API REST

6.3 Contrôleurs

6.4 Services métier

6.5 ORM / Prisma

6.6 Sécurité API

6.7 Gestion des erreurs

7. Développement frontend

7.1 Utilisation de React

7.2 Composants réutilisables

7.3 Gestion état et des appels API

7.4 Gestion formulaires

7.5 Conception de l'interface desktop

7.6 Sécurité frontend

8. Qualité et tests

8.1 Stratégie de tests

8.2 Tests unitaires

8.3 Tests API REST

8.4 Tests de validation et de sécurité

8.5 Tests de l'environnement Docker

8.6 Cahier de recette

9. Conteneurisation et déploiement

9.1 Architecture Docker

9.2 Docker Compose

9.3 Variables d'environnement

9.4 Reverse proxy Traefik

9.5 Healthchecks et fiabilité des services

9.6 Procédure de déploiement de l'environnement

9.7 Qualité et maintenabilité du code

10. Éco-conception et numérique responsable

11. Conclusion

Difficultés rencontrées

Compétences acquises

Améliorations possibles

Introduction

Dans le cadre de ma formation au titre de Concepteur Développeur d'Applications (CDA), j'ai eu l'opportunité de concevoir et développer une application web nommée **TaskFlow**, une plateforme de gestion de tâches inspirée du fonctionnement des tableaux Kanban.

Ce projet est né d'un besoin d'organisation et de suivi des tâches aussi bien dans un contexte personnel que professionnel. De nombreuses solutions existantes proposent des outils de gestion complexes ou peu adaptés aux besoins simples de planification quotidienne. L'objectif de TaskFlow est donc de proposer une application intuitive, moderne et sécurisée permettant aux utilisateurs de créer, organiser, modifier et suivre leurs tâches efficacement.

L'application repose sur une architecture web moderne organisée en couches, avec un frontend développé en React, un backend Node.js / Express et une base de données PostgreSQL. Ce projet m'a permis de mettre en pratique plusieurs compétences liées au développement d'applications sécurisées, notamment la conception d'architecture logicielle, le développement frontend et backend, la gestion d'une base de données relationnelle, la sécurisation des échanges ainsi que le déploiement d'une application web.

Ce dossier présente les différentes étapes de réalisation du projet, depuis l'analyse des besoins jusqu'au déploiement de l'application. Il détaille les choix techniques effectués, les solutions mises en œuvre ainsi que les compétences acquises au cours du développement.

1. Contexte et Analyse des besoins

1.1 Contexte du projet

Le projet TaskFlow est une application web de gestion de tâches permettant aux utilisateurs de créer, organiser, modifier et supprimer leurs tâches de manière sécurisée.

Ce projet s'inscrit dans une démarche de développement d'applications sécurisées respectant les bonnes pratiques du développement web moderne.

1.2 Objectif

L'objectif principal est de fournir une application permettant :

- La gestion des tâches personnelles
- L'organisation du travail utilisateur
- La sécurisation des données

1.2 Problématique

Dans un contexte personnel ou professionnel, la gestion des tâches et le suivi des activités peuvent rapidement devenir complexes lorsque les informations sont dispersées ou mal organisées. De nombreux utilisateurs rencontrent des difficultés pour visualiser l'avancement de leurs tâches, prioriser leur travail et maintenir une organisation efficace au quotidien.

Les solutions existantes sont parfois trop complexes pour des besoins simples de gestion de tâches, ou ne proposent pas une expérience utilisateur suffisamment intuitive et fluide. Il devient alors nécessaire de disposer d'un outil accessible, moderne et ergonomique permettant d'organiser facilement les activités et de suivre leur évolution.

Le projet TaskFlow répond à cette problématique en proposant une application web de gestion de tâches inspirée du fonctionnement Kanban. L'objectif est de permettre aux utilisateurs de créer, organiser, modifier et suivre leurs tâches à travers une interface simple et sécurisée.

Ce projet répond également à des problématiques techniques liées au développement d'applications modernes, notamment :

- La mise en place d'une architecture web multicouche ;
- La sécurisation des données utilisateurs ;
- La gestion des échanges entre frontend et backend ;
- La structuration et la persistance des données dans une base relationnelle ;
- La maintenabilité et l'évolutivité de l'application.

1.3 Public cible

- Utilisateurs particuliers
- Etudiants

1.4 Besoins fonctionnels

L'application doit permettre :

- Création d'un compte utilisateur
- Connexion sécurisée
- Gestion du profil utilisateur
- Modification de l'adresse email
- Modification du mot de passe
-
- Création de tableaux Kanban
- Consultation des tableaux

- Gestion des projets
-
- Gestion des colonnes
- Ajout de colonnes
-
- Création de tâches
- Modification de tâches
- Suppression de tâches
- Déplacement des tâches entre les colonnes
-
- Consultation des projets terminés
-
- Consultation des statistiques
- Nombre total de tâches
- Nombre de tâches terminées
- Taux de complétion
-
- Déconnexion utilisateur

1.5 Besoins non fonctionnels

- Sécurité des données
- Performance des requêtes
- Interface responsive
- Accessibilité

1.6 Contraintes

Techniques :

- Architecture web en couches
- API REST

Sécurité :

- Mot de passe hashé
- Authentification JWT
- Protection des routes

2. Gestion de projet

2.1 Méthodologie Agile

Le projet TaskFlow a été réalisé en suivant une méthodologie Agile afin de favoriser une organisation flexible et progressive du développement. Cette approche a permis de découper le projet en plusieurs fonctionnalités développées étape par étape, tout en facilitant le suivi de l'avancement.

Les fonctionnalités prioritaires ont été identifiées dès le début du projet afin de construire progressivement une version fonctionnelle de l'application. Cette organisation a permis d'améliorer la gestion du temps, d'anticiper les difficultés techniques et de corriger rapidement les problèmes rencontrés durant le développement.

L'approche Agile a également facilité l'évolution du projet en permettant l'ajout progressif de nouvelles fonctionnalités et l'amélioration continue de l'interface utilisateur.

2.2 Organisation et suivi du projet avec Trello

Le suivi du projet TaskFlow a été réalisé avec l'outil Trello selon une organisation inspirée de la méthode Kanban.

Les fonctionnalités du projet sont organisées sous forme de User Stories. Chaque carte Trello représente un besoin fonctionnel ou une évolution technique nécessaire au développement de l'application.

Le tableau Trello est organisé en cinq colonnes principales :

- Backlog : regroupe les fonctionnalités identifiées mais qui ne sont pas encore suffisamment préparées pour être développées ;
- Ready : contient les User Stories qui respectent les critères définis dans la Definition of Ready et qui peuvent être prises en charge ;
- In Progress : regroupe les fonctionnalités en cours de développement ;
- On Approval : contient les fonctionnalités développées qui doivent encore être testées ou validées ;
- Done : regroupe les fonctionnalités terminées et respectant les critères définis dans la Definition of Done.

Chaque User Story contient une description du besoin utilisateur ainsi que des critères d'acceptation permettant de vérifier le comportement attendu de la fonctionnalité.

Cette organisation permet de visualiser rapidement l'état d'avancement du projet et de suivre le cycle de vie d'une fonctionnalité depuis son identification dans le Backlog jusqu'à sa validation dans la colonne Done.

Trello a également été utilisé comme support de suivi du projet. Des captures du tableau et de certaines User Stories sont présentées dans le dossier afin d'illustrer l'organisation et l'évolution du développement de TaskFlow.

2.3 Gestion Git/GitHub

Git et GitHub ont été utilisés afin d'assurer le versionnement du code source et le suivi des différentes évolutions du projet.

Chaque fonctionnalité importante a été développée sur une branche dédiée avant d'être fusionnée dans la branche principale du projet. Cette méthode permet :

- De sécuriser le développement ;
- D'éviter les conflits de code ;
- De conserver un historique des modifications ;
- De faciliter les retours arrière en cas d'erreur.

GitHub a également permis de centraliser le code source et de sauvegarder le projet en ligne.

2.4 Planification des tâches

La planification du projet a été réalisée en plusieurs étapes afin d'organiser efficacement le développement de l'application.

Les fonctionnalités ont été priorisées selon leur importance :

1. Mise en place de l'environnement de développement ;
2. Analyse des besoins fonctionnels et techniques ;
3. Conception de la base de données ;
4. Développement de l'API REST avec Node.js et Express ;
5. Intégration de Prisma ORM et de PostgreSQL ;
6. Développement de l'interface utilisateur avec React ;
7. Développement de la gestion des tableaux Kanban, des colonnes et des tâches ;
8. Mise en place de l'authentification et de la protection des routes ;
9. Développement des fonctionnalités de profil et de statistiques ;
10. Réalisation des tests et correction des anomalies ;
11. Conteneurisation de l'application avec Docker ;
12. Mise en place de Traefik comme reverse proxy ;
13. Mise en place des mécanismes de sauvegarde et de restauration de la base de données ;
14. Préparation du déploiement et de la documentation technique.

Les fonctionnalités ont été priorisées selon leur importance pour le fonctionnement de l'application.

3. Maquettage et conception

3.1 Wireframes

Avant le développement de l'application, une phase de conception a été réalisée afin de définir l'organisation générale des différentes interfaces utilisateur. Des wireframes ont été conçus pour visualiser la structure des pages et préparer l'expérience utilisateur avant l'intégration graphique.

Ces wireframes concernent principalement :

- La page d'inscription ;
- La page de connexion ;
- Le tableau Kanban ;
- La gestion des tableaux ;
- La gestion des colonnes ;
- La gestion des tâches.

Cette étape a permis d'anticiper la navigation dans l'application et de structurer les différents composants de l'interface avant le développement frontend.

3.2 Maquettes Figma

Les maquettes graphiques ont été réalisées avec Figma afin de concevoir une interface moderne, intuitive et responsive. Cette phase de conception visuelle a permis de définir l'apparence globale de l'application ainsi que l'organisation des différents éléments graphiques.

Les maquettes représentent notamment :

- Les formulaires d'authentification ;
- L'organisation des tableaux Kanban ;
- Les colonnes de tâches ;
- Les fonctionnalités de gestion des tâches ;
- L'adaptation de l'interface sur différents supports (ordinateur et mobile).

L'utilisation de Figma a permis de valider l'expérience utilisateur et de préparer plus efficacement l'intégration frontend avec React.

3.3 User Flow

Le User Flow permet de représenter le parcours principal d'un utilisateur au sein de l'application TaskFlow.

Le parcours principal de l'utilisateur est organisé de la manière suivante :

Inscription de l'utilisateur



Connexion à l'application



Accès au tableau de bord



Consultation des tableaux existants



Création ou ouverture d'un tableau Kanban



Consultation des colonnes du tableau



Création d'une tâche



Modification ou déplacement de la tâche entre les colonnes



Consultation de l'avancement du projet



Consultation des statistiques



Gestion du profil utilisateur



Déconnexion

Lorsqu'un utilisateur crée ou ouvre un tableau, il peut organiser ses tâches dans les différentes colonnes du Kanban.

Les tâches peuvent être créées, modifiées, supprimées et déplacées entre les colonnes afin de représenter leur état d'avancement.

L'utilisateur peut également consulter les statistiques de son activité, notamment le nombre total de tâches, le nombre de tâches terminées et le taux de complétion.

Ce parcours permet de représenter les principales interactions entre l'utilisateur et TaskFlow ainsi que la logique générale de navigation dans l'application.

3.4 Diagramme de cas d'utilisation UML

Le diagramme de cas d'utilisation représente les principales interactions entre l'utilisateur et les fonctionnalités proposées par l'application TaskFlow.

Un visiteur non authentifié peut :

- Créer un compte utilisateur ;
- Se connecter à l'application.

Une fois authentifié, l'utilisateur peut :

- Consulter son tableau de bord ;
- Consulter ses tableaux Kanban ;
- Créer un tableau ;
- Gérer les tableaux ;
- Consulter les colonnes d'un tableau ;
- Gérer les tâches ;
- Consulter les projets terminés ;
- Consulter les statistiques ;
- Gérer son profil ;
- Modifier son adresse email ;
- Modifier son mot de passe ;
- Se déconnecter.

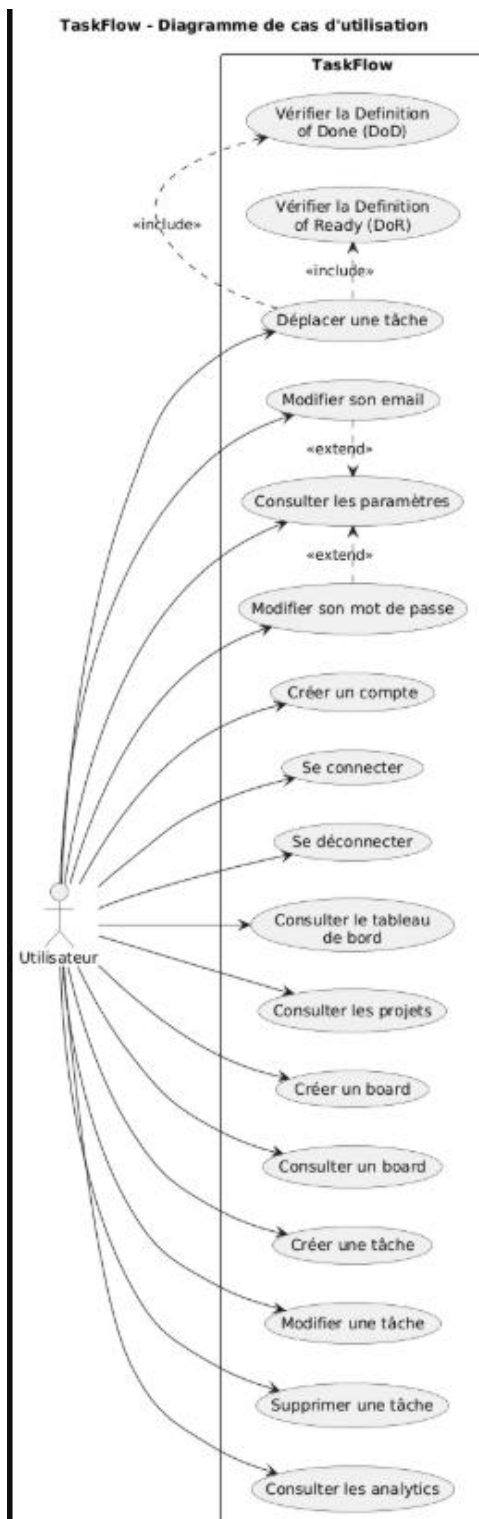
La gestion des tableaux comprend principalement la création et la consultation des tableaux associés à l'utilisateur.

La gestion des tâches comprend :

- La création d'une tâche ;
- La modification d'une tâche ;
- Le déplacement d'une tâche entre les colonnes ;
- La suppression d'une tâche.

Les fonctionnalités liées au tableau de bord permettent également à l'utilisateur de consulter des indicateurs tels que le nombre total de tâches, le nombre de tâches terminées et le taux de complétion.

Ce diagramme permet de définir le périmètre fonctionnel de l'application et de distinguer les fonctionnalités accessibles à un visiteur de celles nécessitant une authentification.



3.5 Diagramme de séquence UML

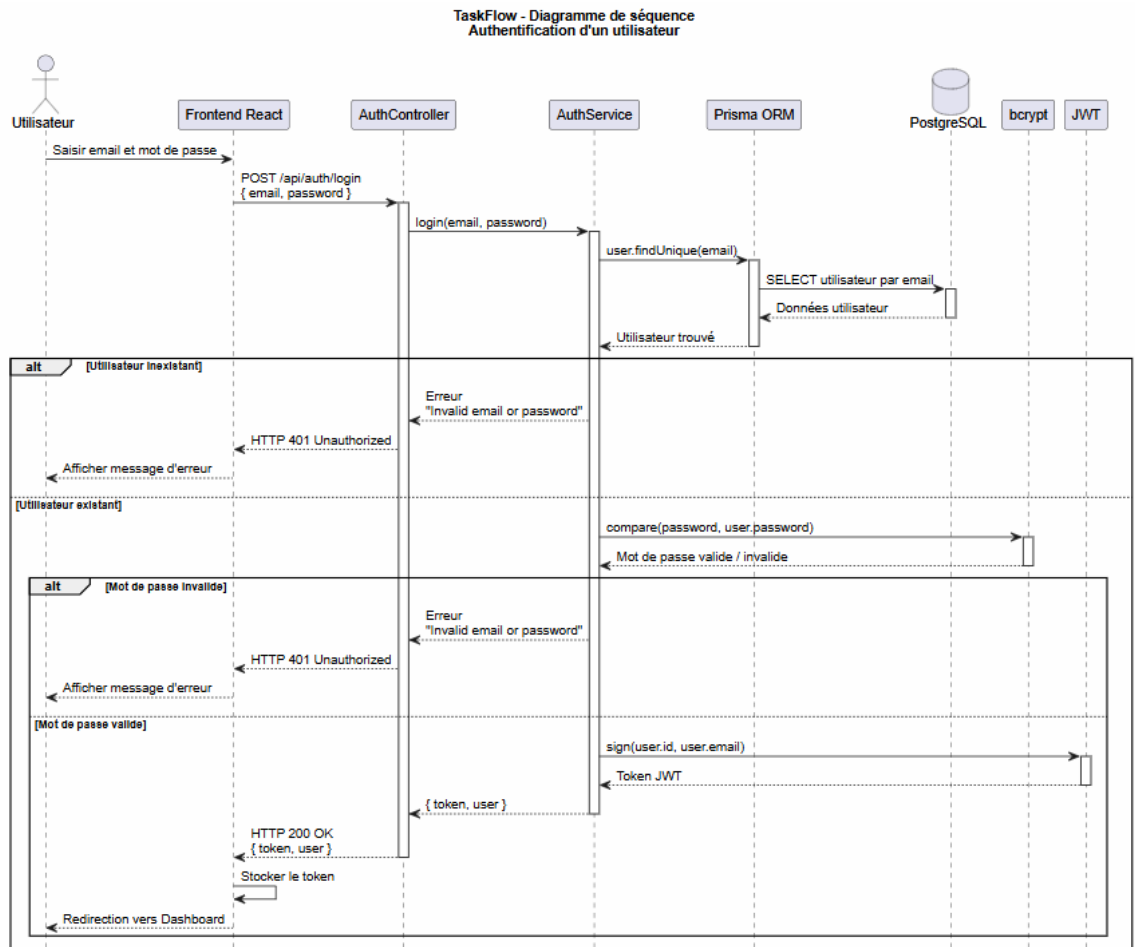
Le diagramme de séquence illustre les échanges entre les différentes couches de l'application lors d'une action utilisateur. Dans le cadre du projet TaskFlow, un scénario de création de tâche a été modélisé.

Le processus fonctionne de la manière suivante :

- L'utilisateur déclenche la création d'une tâche depuis l'interface React ;
- Le frontend prépare les données de la tâche et envoie une requête HTTP POST vers l'API ;
- La requête est reçue par Traefik, qui agit comme reverse proxy ;
- Traefik analyse la règle de routage associée au domaine de l'API et transmet la requête au conteneur Docker du backend Express ;
- Le backend reçoit la requête ;
- Le middleware d'authentification vérifie la présence et la validité du token JWT ;
- Lorsque l'utilisateur est authentifié, le contrôleur transmet les données au service métier chargé de la gestion des tâches ;
- Le service traite les données et utilise Prisma ORM pour communiquer avec PostgreSQL ;
- PostgreSQL enregistre la nouvelle tâche ;
- Prisma retourne les données de la tâche créée au service ;
- L'api retourne une réponse HTTP 201 Created ;
- La réponse transite par Traefik avant d'être retournée au frontend ;
- React actualise l'état de l'interface afin d'afficher la nouvelle tâche dans le tableau Kanban.

Ce diagramme met en évidence la séparation des responsabilités entre le frontend, le reverse proxy, l'API, les middlewares, la couche métier et la couche de persistance.

Il permet également d'illustrer le rôle de Traefik comme point d'entrée HTTP de l'application ainsi que la sécurisation des routes de l'API à l'aide de l'authentification JWT.



3.6 Diagramme de classes UML

Le diagramme de classes représente les principales entités métier de l'application ainsi que leurs relations.

Les classes principales identifiées dans TaskFlow sont :

- User ;
- Board ;
- Column ;
- Task.

Les relations entre les différentes classes permettent de structurer l'organisation métier de l'application :

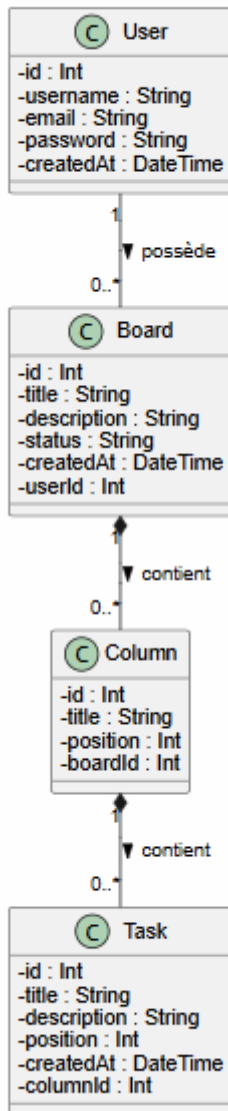
- Un utilisateur peut posséder plusieurs tableaux ;
- Un tableau contient plusieurs colonnes ;
- Une colonne contient plusieurs tâches.

Cette modélisation permet de garantir :

- Une meilleure organisation du code ;

- Une séparation claire des responsabilités ;
- Une meilleure maintenabilité ;
- Une évolution plus simple de l'application dans le futur.

TaskFlow - Diagramme de classes



4. Architecture logicielle

4.1 Architecture globale

L'application TaskFlow repose sur une architecture web multicouche organisée autour de plusieurs composants distincts.

Cette organisation permet de séparer les responsabilités techniques, de faciliter la maintenance du code et de rendre l'application plus évolutive.

L'architecture globale de TaskFlow se compose des éléments suivants :

- Un frontend développé avec React ;
- Une API REST développée avec Node.js et Express ;
- Une couche métier organisée sous forme de services ;
- Des middlewares chargés notamment de l'authentification et de la gestion des erreurs ;
- Prisma ORM pour l'accès aux données ;
- Une base de données relationnelle PostgreSQL ;
- Docker pour la conteneurisation des différents composants ;
- Docker Compose pour l'orchestration des services ;
- Traefik utilisé comme reverse proxy et point d'entrée HTTP de l'application.

Le frontend React est responsable de l'interface utilisateur. Il permet notamment d'afficher les tableaux Kanban, les colonnes, les tâches, les statistiques et les différentes interfaces de gestion du compte utilisateur.

Le frontend communique avec l'API REST à l'aide de requêtes HTTP au format JSON.

Traefik constitue le point d'entrée HTTP de l'application. Il analyse les règles de routage configurées dans l'environnement Docker et transmet les requêtes vers le service correspondant.

Les requêtes destinées à `app.localhost` sont dirigées vers le conteneur du frontend React.

Les requêtes destinées à `api.localhost` sont dirigées vers le conteneur du backend Express.

Le backend expose les différentes routes de l'API REST. Les contrôleurs reçoivent les requêtes HTTP et transmettent les traitements aux services métier.

Les services contiennent la logique applicative et utilisent Prisma ORM afin de communiquer avec PostgreSQL.

La base de données PostgreSQL assure la persistance des utilisateurs, des tableaux, des colonnes et des tâches.

Les différents composants sont exécutés dans des conteneurs Docker distincts et communiquent à travers le réseau interne défini par Docker Compose.

Cette architecture permet d'isoler les différents services tout en conservant une application monolithique organisée en couches.

4.2 Architecture multicouche

L'application TaskFlow suit une architecture multicouche permettant de séparer les différentes responsabilités techniques du projet.

Couche présentation — Frontend React

La couche présentation correspond à l'interface utilisateur développée avec React.

Elle permet :

- L'affichage des données ;
- La gestion des interactions utilisateur ;
- La navigation entre les différentes pages ;
- La gestion de l'état de l'interface ;
- L'envoi des requêtes HTTP vers l'API.

Les appels vers l'API sont centralisés dans des services frontend dédiés à l'authentification, aux tableaux, aux colonnes, aux tâches, aux utilisateurs et aux statistiques.

Couche de routage — Traefik

Traefik agit comme reverse proxy et constitue le point d'entrée HTTP de l'application.

Il analyse les règles de routage et redirige les requêtes vers le service Docker correspondant.

Cette couche permet de centraliser l'accès aux différents services et d'éviter l'exposition directe de certains composants de l'application.

Couche API et contrôleurs — Express

La couche API est développée avec Node.js et Express.

Les routes définissent les différents points d'entrée de l'API REST.

Les contrôleurs permettent :

- De recevoir les requêtes HTTP ;
- De récupérer les données envoyées par le client ;
- D'appeler les services métier ;
- De retourner les réponses HTTP au format JSON.

Couche middleware

Les middlewares assurent plusieurs traitements transversaux.

Ils permettent notamment :

- La vérification des tokens JWT ;
- La protection des routes privées ;
- Le contrôle de certaines données envoyées ;
- La gestion centralisée des erreurs.

Couche métier — Services

Les services contiennent la logique métier de l'application.

Ils permettent notamment de gérer :

- L'authentification des utilisateurs ;
- Les tableaux Kanban ;
- Les colonnes ;
- Les tâches ;
- Les informations utilisateur ;
- Les statistiques de l'application.

Cette séparation permet d'éviter de placer directement la logique métier dans les contrôleurs.

Couche d'accès aux données — Prisma ORM

Prisma est utilisé comme ORM afin de gérer la communication entre le backend et PostgreSQL.

Il permet :

- d'effectuer les opérations CRUD ;

- de manipuler les relations entre les entités ;
- de centraliser les accès aux données ;
- de simplifier les requêtes vers PostgreSQL.

Couche de persistance — PostgreSQL

PostgreSQL assure le stockage des données de l'application.

La base de données contient principalement les informations relatives :

- Aux utilisateurs ;
- Aux tableaux ;
- Aux colonnes ;
- Aux tâches.

Cette architecture multicouche permet d'améliorer la lisibilité du code, la séparation des responsabilités, la maintenabilité et l'évolutivité de TaskFlow.

4.3 Choix de l'architecture monolithique

Pour le projet TaskFlow, une architecture monolithique organisée en couches a été retenue.

Le projet étant une application web de gestion de projets et de tâches, une architecture monolithique répond pleinement aux besoins tout en limitant la complexité technique.

Cette solution présente plusieurs avantages :

- Simplicité de développement et de maintenance ;
- Communication directe entre les composants ;
- Mise en œuvre rapide ;
- Évolution facilitée.

Le backend est développé avec Node.js et Express et s'appuie sur une séparation des responsabilités entre les routes, les contrôleurs, les services, les middlewares et Prisma ORM.

L'application est conteneurisée avec Docker afin d'isoler le frontend, le backend, PostgreSQL et Traefik, tout en conservant une architecture applicative monolithique.

4.4 Alternatives étudiées

Deux architectures ont été étudiées avant de retenir l'architecture monolithique.

Architecture microservices

Les microservices permettent de découper une application en plusieurs services indépendants, offrant une meilleure scalabilité et une plus grande autonomie de développement.

Cependant, cette architecture nécessite une infrastructure plus complexe, des mécanismes de communication entre services et une supervision plus importante. Pour un projet de la taille de TaskFlow, cette solution était surdimensionnée.

Architecture hexagonale

L'architecture hexagonale favorise le découplage entre la logique métier et les technologies utilisées, ce qui améliore la testabilité et l'évolutivité.

Toutefois, elle introduit davantage de couches techniques et de complexité. Une architecture multicouche classique a donc été privilégiée, car elle répondait pleinement aux besoins du projet.

4.5 Stratégie de sécurité

La sécurité a été intégrée dès la conception de TaskFlow afin de protéger les utilisateurs, les données et les échanges entre les différents composants.

Authentification

L'authentification repose sur des JSON Web Tokens (JWT). Après connexion, un token est généré puis utilisé pour accéder aux routes protégées de l'API grâce à un middleware de vérification.

Protection des données

Les mots de passe sont hachés avec bcrypt avant leur enregistrement dans PostgreSQL. Les données reçues par l'API sont validées avec Express Validator afin de limiter les erreurs et les tentatives d'exploitation.

Les routes sensibles sont protégées et un middleware centralise la gestion des erreurs en retournant des codes HTTP adaptés.

Sécurisation de l'environnement

Les informations sensibles (connexion à PostgreSQL, clé JWT, ports) sont stockées dans des variables d'environnement afin de ne pas être présentes dans le code source.

L'application est exécutée dans des conteneurs Docker distincts (frontend, backend, PostgreSQL et Traefik). La base de données n'est pas exposée directement et toutes les requêtes transitent par Traefik, configuré comme reverse proxy.

Des healthchecks Docker vérifient automatiquement la disponibilité des services afin d'améliorer la fiabilité de l'environnement.

Sauvegarde

Une procédure de sauvegarde et de restauration de la base PostgreSQL a été mise en place à l'aide de `pg_dump` et `pg_restore`, garantissant la récupération des données en cas d'incident.

Évolutions possibles

Dans un environnement de production, la sécurité pourrait être renforcée par l'utilisation de HTTPS avec des certificats TLS, l'ajout d'en-têtes de sécurité HTTP, une limitation du nombre de requêtes ainsi que l'automatisation des sauvegardes.

5. Base de données

La base de données constitue la couche de persistance de TaskFlow.

Elle permet de conserver les informations relatives aux utilisateurs ainsi que les données nécessaires à la gestion des tableaux Kanban.

Le système de gestion de base de données PostgreSQL a été utilisé avec Prisma ORM afin de faciliter les échanges entre l'API Express et la base de données.

5.1 Choix PostgreSQL

PostgreSQL a été retenu comme système de gestion de base de données relationnelle pour le projet TaskFlow en raison de sa robustesse, de sa fiabilité et de sa bonne gestion des relations entre les données.

Cette solution s'intègre naturellement avec Prisma ORM, utilisé dans le backend Node.js pour simplifier les opérations CRUD et la gestion des relations entre les entités.

Le modèle de données de TaskFlow repose sur quatre entités principales : **User**, **Board**, **Column** et **Task**, liées entre elles afin de représenter l'organisation d'un tableau Kanban.

5.2 Modèle Conceptuel de Données (MCD)

Le MCD représente les principales entités métier de TaskFlow ainsi que leurs relations.

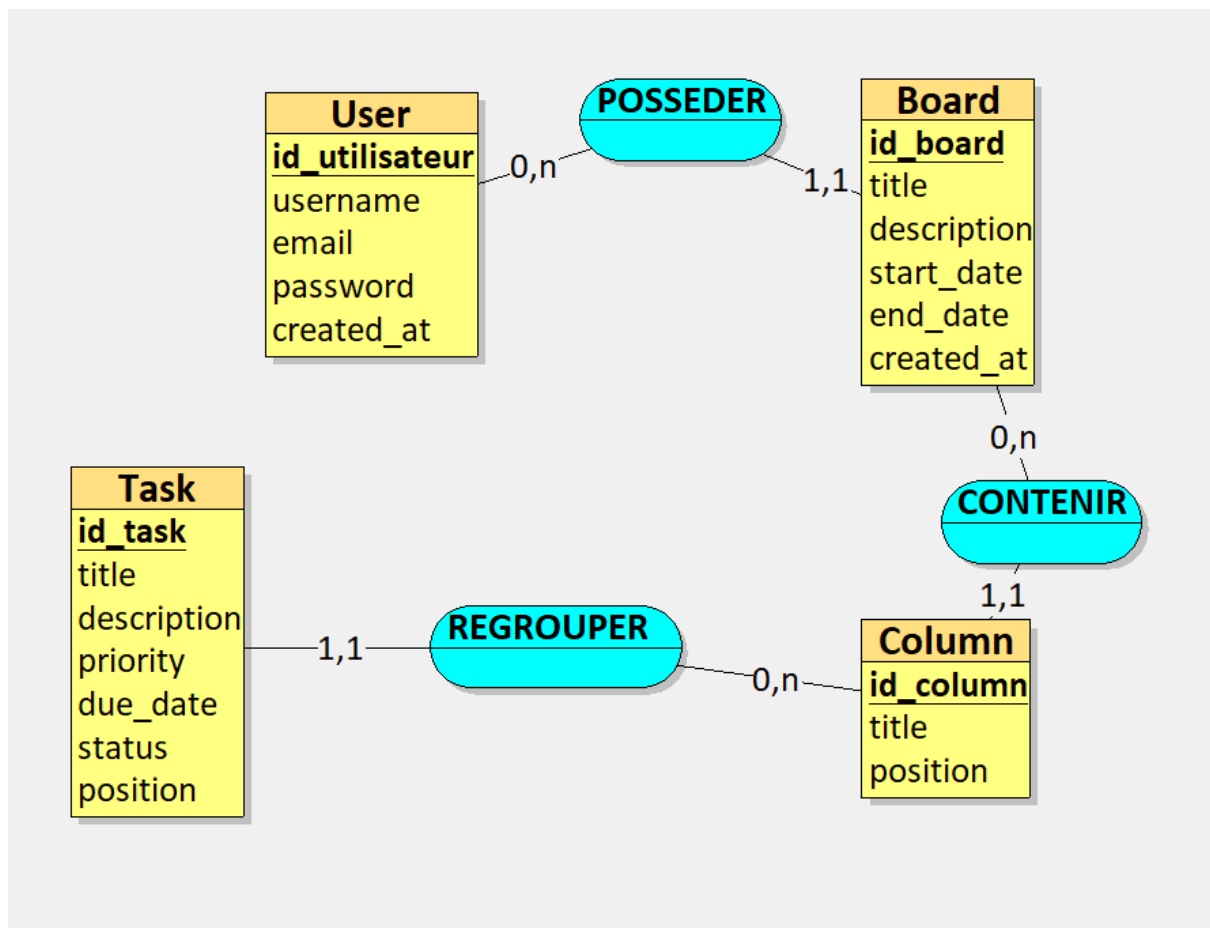
Les quatre entités identifiées sont :

- **User** : utilisateur de l'application ;
- **Board** : tableau Kanban ;
- **Column** : colonne d'un tableau ;
- **Task** : tâche appartenant à une colonne.

Les relations sont les suivantes :

- un utilisateur possède plusieurs tableaux (0,N) ;
- un tableau contient plusieurs colonnes (0,N) ;
- une colonne contient plusieurs tâches (0,N).

Le MCD décrit uniquement la logique métier sans tenir compte des contraintes techniques liées à la base de données.



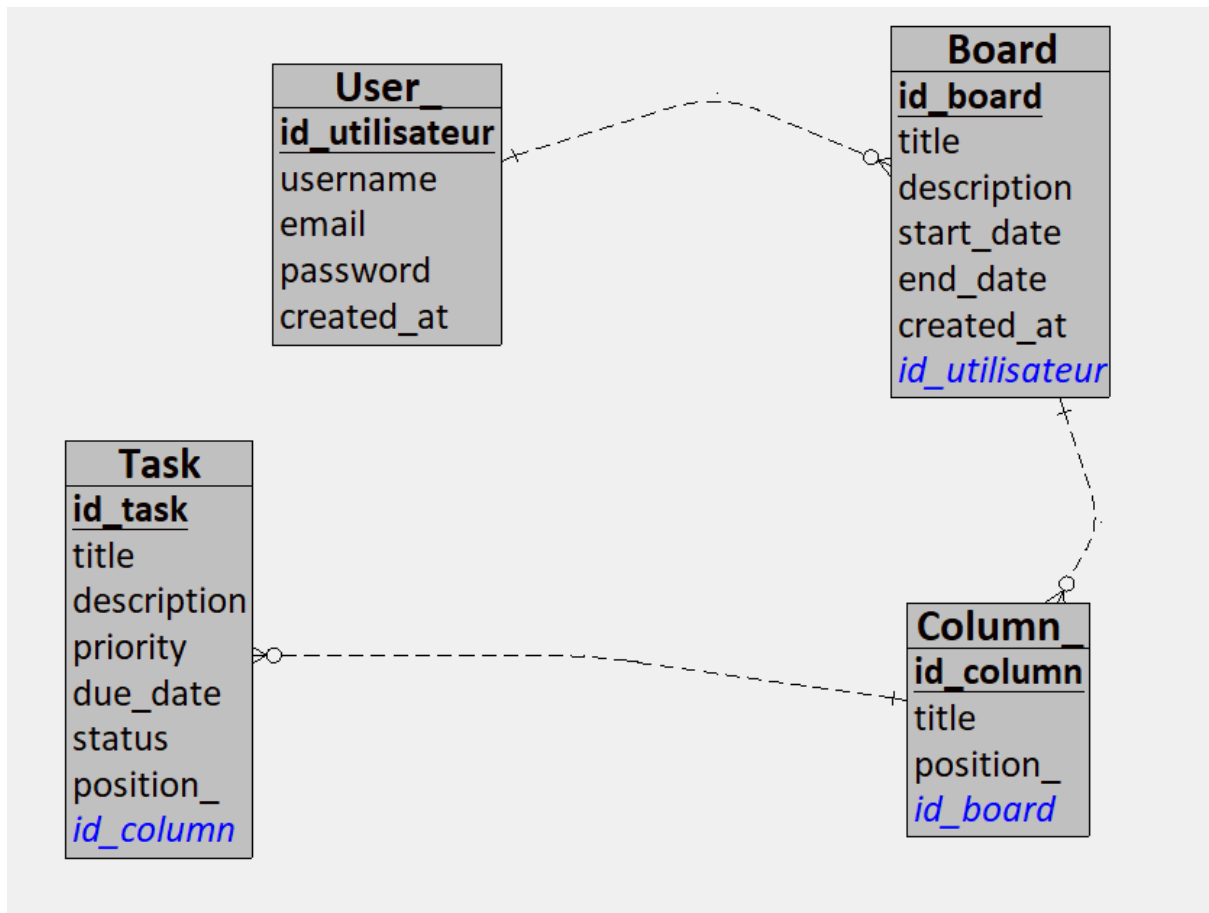
5.3 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en une structure relationnelle compatible avec PostgreSQL.

Il est composé de quatre tables principales :

- **User** (*id*, *username*, *email*, *password*, *createdAt*) ;
- **Board** (*id*, *title*, *description*, *startDate*, *endDate*, *createdAt*, *userId*) ;
- **Column** (*id*, *title*, *position*, *boardId*) ;
- **Task** (*id*, *title*, *description*, *priority*, *dueDate*, *status*, *position*, *columnId*).

Les relations sont assurées par les clés étrangères **userId**, **boardId** et **columnId**.



5.4 Relations

Les relations entre les tables permettent de représenter l'organisation des données de TaskFlow :

- **User 1 → N Board**
- **Board 1 → N Column**
- **Column 1 → N Task**

Ces relations sont définies dans le schéma Prisma et appliquées automatiquement dans PostgreSQL grâce aux migrations.

Prisma permet ensuite au backend de manipuler les entités et leurs relations à partir des services de l'application.

5.5 Intégrité référentielle

L'intégrité référentielle permet de garantir la cohérence des données stockées dans PostgreSQL.

Dans TaskFlow, plusieurs mécanismes sont utilisés afin de limiter les incohérences dans la base de données.

Clés primaires

Chaque table possède une clé primaire permettant d'identifier de manière unique chaque enregistrement.

Les principales clés primaires sont :

- `User.id` ;
- `Board.id` ;
- `Column.id` ;
- `Task.id`.

Ces identifiants permettent de distinguer chaque ressource enregistrée dans l'application.

Clés étrangères

Les clés étrangères permettent de créer les relations entre les différentes tables.

TaskFlow utilise notamment :

- `Board.userId` pour associer un tableau à un utilisateur ;
- `Column.boardId` pour associer une colonne à un tableau ;
- `Task.columnId` pour associer une tâche à une colonne.

Ces relations permettent de conserver une organisation cohérente des données.

Contraintes d'unicité

L'adresse email d'un utilisateur doit être unique.

Cette contrainte permet d'empêcher la création de plusieurs comptes utilisant la même adresse email.

Une vérification est également réalisée dans le service d'authentification avant la création d'un utilisateur.

Contraintes sur les données obligatoires

Certains champs nécessaires au fonctionnement de l'application ne peuvent pas être absents.

Ces contraintes sont définies dans le modèle de données et complétées par les validations réalisées dans l'API.

Validation côté backend

L'intégrité de la base de données est complétée par des contrôles réalisés dans le backend.

Les données reçues sont vérifiées avant certaines opérations de création ou de modification.

Par exemple, lors de l'inscription d'un utilisateur, le backend vérifie notamment :

- la présence du nom d'utilisateur ;
- la longueur minimale du nom d'utilisateur ;
- la présence de l'adresse email ;
- le format de l'adresse email ;
- la présence du mot de passe ;
- la longueur minimale du mot de passe.

Cette double vérification, au niveau de l'API et de la base de données, permet de renforcer la cohérence des données enregistrées dans TaskFlow.

Gestion des migrations

Les évolutions de la structure de la base de données sont gérées avec les migrations Prisma.

Les migrations permettent de conserver un historique des modifications du schéma et d'appliquer les évolutions de manière contrôlée.

Lors de la mise en place de l'environnement Docker, les migrations existantes ont été appliquées à la base PostgreSQL avec Prisma.

Trois migrations ont notamment été détectées et appliquées avec succès lors de la préparation de l'environnement conteneurisé.

Cette gestion permet de synchroniser la structure attendue par l'application avec la structure de la base de données PostgreSQL.

5.6 Sauvegarde et restauration de la base de données

Une stratégie de sauvegarde et de restauration de la base de données PostgreSQL a été mise en place afin de limiter les risques de perte de données et de vérifier la capacité de récupération de l'application en cas d'incident.

Dans l'environnement Docker de TaskFlow, PostgreSQL est exécuté dans un conteneur dédié nommé `taskflow-db`. Les données de la base sont persistées à l'aide d'un volume Docker.

Le volume permet de conserver les données lorsque le conteneur PostgreSQL est arrêté ou recréé. Cependant, la persistance par volume Docker ne constitue pas à elle seule une stratégie de sauvegarde.

Une sauvegarde logique de la base de données a donc été réalisée avec l'outil `pg_dump` fourni par PostgreSQL.

La commande utilisée est la suivante :

```
docker exec taskflow-db pg_dump -U postgres -d taskflow -F c -f /tmp/taskflow_backup.dump
```

L'option `-F c` permet de générer une archive au format personnalisé de PostgreSQL. Ce format est compatible avec l'outil `pg_restore` et permet de gérer plus facilement les opérations de restauration.

Après la génération de la sauvegarde, la présence du fichier dans le conteneur PostgreSQL a été vérifiée.

La commande suivante a notamment permis de contrôler l'existence et la taille du fichier :

```
docker exec taskflow-db ls -lh /tmp/taskflow_backup.dump
```

Le fichier de sauvegarde généré possédait une taille d'environ 12 Ko au moment du test.

La validité de l'archive peut être contrôlée avec `pg_restore` afin de vérifier que PostgreSQL reconnaît le fichier comme une archive exploitable.

Une procédure de restauration a également été étudiée afin de permettre la récupération des données à partir de la sauvegarde.

Le processus de restauration repose sur les étapes suivantes :

- disposer d'une instance PostgreSQL fonctionnelle ;
- transférer ou rendre accessible le fichier de sauvegarde au conteneur PostgreSQL ;
- créer ou préparer la base de données cible ;
- utiliser `pg_restore` afin de restaurer le contenu de l'archive ;
- vérifier la présence des tables et des données restaurées.

La mise en place de cette procédure permet de répondre à plusieurs risques :

- suppression accidentelle de données ;
- erreur technique lors d'une évolution de la base ;
- corruption ou perte d'une instance PostgreSQL ;
- besoin de recréer un environnement à partir d'une sauvegarde existante.

Dans un environnement de production, cette stratégie devrait être complétée par une automatisation des sauvegardes, une politique de rétention et un stockage des archives sur un emplacement distinct du serveur hébergeant la base de données.

Des tests de restauration réguliers devraient également être réalisés afin de vérifier que les sauvegardes restent exploitables.

Cette démarche permet de ne pas considérer la simple création d'un fichier de sauvegarde comme une garantie suffisante. La capacité à vérifier et à restaurer les données constitue un élément essentiel de la stratégie de continuité des données de TaskFlow.

6. Développement backend

Le backend de TaskFlow constitue le cœur applicatif du projet.

Il expose une API REST permettant au frontend React d'accéder aux différentes fonctionnalités de l'application.

Le backend a été développé avec Node.js et Express. Prisma ORM est utilisé pour l'accès aux données stockées dans PostgreSQL.

L'application backend est organisée selon une architecture multicouche afin de séparer la gestion des requêtes HTTP, la logique applicative, les traitements transversaux et l'accès aux données.

6.1 Organisation du projet

Le backend de TaskFlow a été développé avec **Node.js** et **Express** selon une architecture organisée en couches afin de séparer les responsabilités et de faciliter la maintenance.

Le projet est structuré autour des dossiers suivants :

- **routes** : définition des routes de l'API REST ;
- **controllers** : traitement des requêtes HTTP et construction des réponses ;
- **services** : logique métier de l'application ;
- **middlewares** : authentification, validation et gestion des erreurs ;
- **config** : configuration des composants techniques ;
- **prisma** : schéma de données et migrations PostgreSQL.

Cette organisation améliore la lisibilité du code, facilite son évolution et favorise la réutilisation des traitements.

Le frontend React communique avec le backend via une API REST au format JSON. Dans l'environnement Docker, toutes les requêtes transitent par **Traefik**, qui assure le rôle de reverse proxy.

6.2 Routes API REST

Le backend expose une API REST permettant de gérer les principales fonctionnalités de TaskFlow.

Les routes sont organisées par domaine fonctionnel :

- **Authentification** : `/api/auth`
- **Tableaux Kanban** : `/api/boards`
- **Colonnes** : `/api/columns`
- **Tâches** : `/api/tasks`
- **Utilisateurs** : `/api/users`
- **Statistiques** : `/api/analytics`

Une route `/health` est également disponible afin de vérifier l'état du backend. Elle est utilisée par Docker et Traefik pour contrôler la disponibilité du service.

6.3 Contrôleurs

Les contrôleurs assurent l'interface entre les routes et les services métier.

Ils récupèrent les données des requêtes, appellent le service correspondant puis retournent une réponse HTTP au format JSON.

Cette séparation permet de conserver des contrôleurs légers et de centraliser la logique métier dans les services.

6.4 Services métier

Les services regroupent la logique métier de l'application et utilisent **Prisma ORM** pour accéder à PostgreSQL.

Ils gèrent notamment :

- l'authentification des utilisateurs ;
- la gestion des tableaux Kanban ;
- la gestion des colonnes ;
- la gestion des tâches ;
- le profil utilisateur ;
- les statistiques.

Le service d'authentification assure le contrôle des utilisateurs, le hachage des mots de passe avec **bcrypt** et la génération des **JSON Web Tokens (JWT)**.

Cette organisation facilite la maintenance, la réutilisation du code et les tests unitaires.

6.5 ORM / Prisma

Prisma ORM est utilisé comme couche d'accès aux données entre le backend et PostgreSQL.

Il permet de réaliser les opérations CRUD, de manipuler les relations entre les entités et de limiter l'écriture de requêtes SQL.

Le fichier `schema.prisma` décrit les modèles **User**, **Board**, **Column** et **Task**, leurs relations ainsi que les contraintes (unicité, valeurs par défaut et suppressions en cascade).

Le **Prisma Client**, généré automatiquement à partir du schéma, est utilisé dans les services pour communiquer avec la base de données.

Les évolutions de la structure sont gérées grâce aux **migrations Prisma**, appliquées automatiquement lors du déploiement de l'application.

6.6 Sécurité API

Le backend de TaskFlow intègre plusieurs mécanismes de sécurité afin de protéger les données et les fonctionnalités de l'application.

L'authentification repose sur **JSON Web Token (JWT)**. Lorsqu'un utilisateur se connecte avec des identifiants valides, un token est généré puis utilisé pour accéder aux routes protégées de l'API.

Les mots de passe sont **hachés avec bcrypt** avant leur enregistrement dans PostgreSQL. Lors de la connexion, le mot de passe saisi est comparé au hash stocké dans la base de données.

Les routes privées sont protégées par un **middleware Express** qui vérifie la validité du token avant d'autoriser l'accès aux ressources.

Les données reçues par l'API sont contrôlées avec **Express Validator** afin de vérifier les champs obligatoires, le format des adresses e-mail et les règles de validation. En cas d'erreur, l'API retourne une réponse **HTTP 400** sans exécuter la logique métier.

Les informations sensibles, telles que **DATABASE_URL** et **JWT_SECRET**, sont stockées dans des variables d'environnement afin de ne pas être intégrées au code source.

Enfin, le backend communique avec PostgreSQL via **Prisma ORM** sur le réseau interne Docker. La base de données n'est pas exposée directement à l'extérieur, ce qui renforce la sécurité de l'architecture.

6.7 Gestion des erreurs

La gestion des erreurs est centralisée grâce à un **middleware Express** chargé de retourner des réponses HTTP adaptées et de limiter l'exposition d'informations techniques.

Les erreurs de validation sont interceptées avant l'exécution des contrôleurs grâce à **Express Validator**. Lorsqu'une donnée est invalide, une réponse **HTTP 400 (Bad Request)** est renvoyée au client.

Les principaux codes HTTP utilisés sont :

- **200** : requête traitée avec succès ;
- **201** : ressource créée ;
- **400** : données invalides ;
- **401** : authentification absente ou invalide ;
- **404** : ressource introuvable ;
- **500** : erreur interne du serveur.

Durant le développement, les journaux Docker (**docker logs**) ont facilité l'identification de plusieurs anomalies, notamment une erreur d'authentification entre Prisma et PostgreSQL ainsi qu'une mauvaise configuration du port du backend. L'analyse de ces erreurs a permis de corriger rapidement la configuration et d'améliorer la fiabilité de l'application.

7. Développement frontend

Le frontend de TaskFlow constitue l'interface utilisée par les utilisateurs pour interagir avec l'application.

Il permet notamment de créer un compte, de se connecter, de consulter les tableaux Kanban, de gérer les colonnes et les tâches, de consulter les statistiques et de modifier certaines informations du compte utilisateur.

Le frontend a été développé avec React et communique avec l'API REST Node.js/Express à travers des requêtes HTTP au format JSON.

Dans l'environnement Docker de TaskFlow, l'application frontend est exécutée dans un conteneur dédié. L'accès à l'interface est réalisé à travers Traefik avec le nom d'hôte **app.localhost**.

Les requêtes destinées au backend sont envoyées vers **api.localhost** puis routées par Traefik vers le conteneur Express.

7.1 Utilisation de React

Le frontend de **TaskFlow** a été développé avec **React** afin de concevoir une interface utilisateur dynamique basée sur une architecture en composants.

React a été choisi pour sa modularité, sa facilité de maintenance et son intégration avec une API REST. Son système de composants permet de séparer les différentes parties de l'application et de mettre à jour uniquement les éléments concernés lors d'une modification de l'état.

Cette approche est particulièrement adaptée à une application de type **Kanban**, où les tableaux, colonnes et tâches évoluent fréquemment sans recharger la page.

La navigation est assurée par **React Router**, qui gère les principales vues de l'application :

- inscription ;
- connexion ;
- tableau de bord ;
- tableau Kanban ;
- statistiques ;
- profil utilisateur.

L'application fonctionne ainsi comme une **Single Page Application (SPA)** offrant une navigation fluide entre les différentes pages.

7.2 Composants réutilisables

L'interface est organisée autour de composants React réutilisables, chacun ayant une responsabilité précise.

Les principaux composants représentent :

- les tableaux Kanban ;
- les colonnes ;
- les tâches ;
- la barre latérale ;
- l'en-tête ;
- les fenêtres modales.

Le tableau Kanban est construit selon une hiérarchie simple :

Tableau → Colonne → Tâche

Les données sont transmises entre les composants grâce aux **props**, tandis que certaines actions sont remontées vers les composants parents afin de maintenir une organisation claire du code.

Cette architecture facilite la maintenance, la réutilisation des composants et l'évolution de l'application.

7.3 Gestion de l'état et des appels API

Le frontend utilise le **state React** afin de gérer les données dynamiques de l'application, notamment :

- les formulaires ;
- les tableaux Kanban ;

- les colonnes ;
- les tâches ;
- les fenêtres modales ;
- les informations de l'utilisateur connecté.

Chaque modification de l'état entraîne automatiquement la mise à jour des éléments concernés de l'interface.

Les échanges avec le backend sont centralisés dans un dossier **services**, chaque service étant dédié à un domaine fonctionnel :

- **authService** : authentification ;
- **boardService** : tableaux ;
- **columnService** : colonnes ;
- **taskService** : tâches ;
- **analyticsService** : statistiques ;
- **userService** : profil utilisateur.

Les communications avec l'API REST sont réalisées au format **JSON** via des requêtes HTTP. Cette séparation permet de conserver des composants centrés sur l'interface tandis que les services gèrent la communication avec le backend.

Cette organisation améliore la lisibilité, la réutilisation du code et facilite les évolutions futures de l'application.

7.4 Gestion des formulaires

TaskFlow utilise plusieurs formulaires permettant aux utilisateurs de créer un compte, se connecter, gérer leurs tableaux et leurs tâches.

Les données saisies sont gérées avec le **state React**. Lors de la soumission d'un formulaire, les informations sont envoyées au service correspondant, qui communique avec l'API REST au format JSON.

Des contrôles simples sont réalisés côté frontend afin d'améliorer l'expérience utilisateur. Toutefois, les validations importantes sont également effectuées côté backend avec **Express Validator**, garantissant la fiabilité et la sécurité des données reçues.

Cette approche permet d'offrir une interface fluide tout en conservant les contrôles de sécurité sur le serveur.

7.5 Conception de l'interface desktop

L'interface de TaskFlow a été conçue principalement pour une utilisation sur ordinateur.

Le tableau de bord est composé d'une barre latérale, d'un en-tête et d'une zone principale affichant les tableaux Kanban, les colonnes et les tâches.

La mise en page repose sur **Flexbox** et **CSS Grid**, offrant une interface claire et adaptée à la gestion simultanée de plusieurs colonnes.

Des zones de défilement ont été ajoutées afin de conserver une navigation fluide lorsque le nombre de tableaux ou de tâches devient important.

L'adaptation complète aux supports mobiles constitue une évolution envisageable du projet.

7.6 Sécurité frontend

Le frontend participe au processus d'authentification tout en laissant les contrôles de sécurité au backend.

Après une connexion réussie, le backend retourne un **JSON Web Token (JWT)** utilisé lors des appels aux routes protégées de l'API. La vérification du token est réalisée exclusivement par le backend.

Les échanges entre le frontend et le backend sont réalisés au format **JSON** via l'API REST.

Dans l'environnement Docker, **Traefik** assure le routage des requêtes entre les domaines `app.localhost` et `api.localhost`, tandis que PostgreSQL reste accessible uniquement depuis le réseau interne Docker.

L'environnement de développement fonctionne actuellement en **HTTP**. En production, l'utilisation de **HTTPS** avec un certificat **TLS** permettrait de sécuriser les échanges entre le navigateur et l'application.

Cette organisation garantit une séparation claire des responsabilités : le frontend gère l'interface utilisateur tandis que le backend assure l'authentification, la validation des données et le contrôle des accès.

8. Qualité et tests

La phase de tests a permis de valider les principales fonctionnalités de TaskFlow et de vérifier la fiabilité de l'application.

Les vérifications ont porté sur :

- l'authentification des utilisateurs ;
- les opérations CRUD sur les tableaux, colonnes et tâches ;
- les routes de l'API REST ;
- la validation des données ;
- les échanges entre le frontend, le backend et PostgreSQL ;
- le fonctionnement de l'environnement Docker.

Plusieurs types de tests ont été réalisés :

- tests unitaires avec Jest ;

- tests de l'API REST ;
- tests fonctionnels manuels ;
- tests de l'environnement Docker.

Cette démarche a permis d'identifier et de corriger plusieurs anomalies avant la validation du projet.

8.1 Stratégie de tests

Les tests ont été réalisés progressivement tout au long du développement.

Chaque fonctionnalité a été vérifiée depuis l'interface utilisateur et directement via l'API REST afin de contrôler le comportement du frontend, du backend et de la base de données.

Les journaux Docker ont également été utilisés pour identifier certaines erreurs de configuration et faciliter leur résolution.

Cette approche combine des tests automatisés et des vérifications fonctionnelles afin de garantir le bon fonctionnement global de l'application.

8.2 Tests unitaires

Des tests unitaires ont été développés avec **Jest** afin de valider le service d'authentification.

Les dépendances externes (**Prisma**, **bcrypt** et **jsonwebtoken**) ont été simulées à l'aide de mocks afin de tester uniquement la logique métier.

Les principaux scénarios vérifiés sont :

- création d'un utilisateur ;
- refus d'une inscription avec un email déjà utilisé ;
- hachage du mot de passe ;
- refus d'une connexion invalide ;
- génération d'un token JWT après une authentification réussie.

L'ensemble des tests a été validé avec succès :

- **1 suite de tests exécutée ;**
- **6 tests réussis sur 6.**

Ces tests permettent de limiter les risques de régression lors des évolutions du backend.

8.3 Tests de l'API REST

Les principales routes de l'API ont été testées afin de vérifier leur bon fonctionnement.

Les tests ont concerné :

- l'inscription et la connexion des utilisateurs ;
- la gestion des tableaux ;
- la gestion des tâches ;
- les routes protégées.

Les vérifications ont porté sur les réponses HTTP, les données échangées au format JSON, la validation des données et le contrôle des accès.

Les tests ont été réalisés avec **Postman** ainsi qu'à l'aide de requêtes HTTP exécutées depuis le terminal.

Ils ont notamment permis d'identifier une erreur de configuration entre le backend et PostgreSQL, rapidement corrigée après analyse des journaux Docker.

Ces vérifications ont confirmé le bon fonctionnement de l'API indépendamment du frontend.

8.4 Tests de validation et de sécurité

Des tests ont été réalisés afin de vérifier le comportement de l'API en présence de données invalides ou d'une authentification absente.

Les principaux scénarios testés concernent :

- les champs obligatoires des formulaires ;
- le format des adresses e-mail ;
- la longueur minimale des mots de passe ;
- l'accès aux routes protégées sans token JWT.

Les validations sont assurées côté serveur avec **Express Validator**. En cas d'erreur, l'API retourne une réponse **HTTP 400 (Bad Request)**.

Les tests ont également permis de vérifier que les mots de passe sont correctement hachés avec **bcrypt** avant leur enregistrement dans PostgreSQL.

Ces vérifications garantissent que les contrôles de sécurité ne reposent pas uniquement sur le frontend.

8.5 Tests de l'environnement Docker

L'environnement Docker a été testé afin de vérifier le bon fonctionnement des différents services de l'application.

Les vérifications ont porté sur :

- l'état des conteneurs Docker ;
- les healthchecks du backend et de PostgreSQL ;

- le routage des requêtes par Traefik.

Les tests ont notamment permis d'identifier une erreur de configuration du port du backend, empêchant le healthcheck de fonctionner correctement. Après correction, le backend a retrouvé un état sain et la route `/health` a confirmé le bon fonctionnement du service.

Ces tests ont permis de valider le fonctionnement de l'architecture Docker mise en place pour TaskFlow.

8.6 Cahier de recette

Un cahier de recette fonctionnelle a été utilisé afin de vérifier les principales fonctionnalités de TaskFlow.

Fonctionnalité testée	Résultat attendu	Résultat obtenu
Inscription utilisateur	Création du compte utilisateur	Conforme
Connexion utilisateur	Authentification et génération du JWT	Conforme
Création d'un tableau	Nouveau tableau enregistré	Conforme
Affichage des tableaux	Affichage des tableaux de l'utilisateur	Conforme
Création d'une tâche	Ajout de la tâche dans une colonne	Conforme
Modification d'une tâche	Mise à jour des informations	Conforme
Déplacement d'une tâche	Changement de colonne ou de position	Conforme

Suppression d'une tâche	Suppression de la tâche	Conforme
Accès sans token	Refus de l'accès à une route protégée	Conforme
Données d'inscription invalides	Retour d'une erreur HTTP 400	Conforme
Tests unitaires d'authentification	Six scénarios Jest validés	Conforme
Vérification du backend	Retour de l'état du service	Conforme
Vérification PostgreSQL	Service déclaré sain	Conforme
Routage API avec Traefik	Transmission vers le backend	Conforme

Les résultats du cahier de recette permettent de vérifier le fonctionnement des principales fonctionnalités du MVP.

Les anomalies rencontrées durant les tests ont été identifiées et corrigées progressivement.

Les tests unitaires automatisés complètent les vérifications fonctionnelles en permettant de contrôler automatiquement certaines règles applicatives sensibles du backend.

9. Conteneurisation et déploiement

TaskFlow est conteneurisé avec **Docker** afin de garantir un environnement de développement reproductible et d'isoler les différents composants de l'application.

L'environnement est orchestré avec **Docker Compose** et comprend quatre services principaux :

- Traefik ;

- le frontend React ;
- le backend Node.js / Express ;
- PostgreSQL.

Cette organisation facilite le déploiement, la maintenance et la communication entre les services tout en conservant une architecture monolithique.

9.1 Architecture Docker

L'architecture Docker repose sur le fonctionnement suivant :

Navigateur → Traefik → Frontend React

Navigateur → Traefik → API Express → Prisma → PostgreSQL

Traefik joue le rôle de **reverse proxy** en dirigeant les requêtes vers le frontend ou le backend selon le nom d'hôte utilisé.

Le backend communique avec PostgreSQL via le réseau interne Docker, sans exposer directement la base de données à l'extérieur.

Cette architecture améliore l'isolation des services et facilite leur déploiement.

9.2 Docker Compose

Docker Compose permet d'orchestrer l'ensemble des services de TaskFlow à partir d'un unique fichier de configuration.

Il gère notamment :

- la construction des images Docker ;
- les variables d'environnement ;
- les volumes de données ;
- les dépendances entre les services ;
- les healthchecks ;
- la configuration de Traefik.

Un volume dédié assure la persistance des données PostgreSQL et le backend démarre uniquement lorsque la base de données est disponible..

9.3 Variables d'environnement

La configuration de l'application repose sur des variables d'environnement regroupées dans un fichier `.env.docker`.

Elles permettent notamment de définir :

- les paramètres de connexion à PostgreSQL ;
- la variable `DATABASE_URL` utilisée par Prisma ;
- le secret JWT ;
- le port du backend.

Cette approche évite d'intégrer des informations sensibles dans le code source et facilite le changement d'environnement.

9.4 Reverse proxy Traefik

Traefik assure le routage des requêtes HTTP entre le navigateur, le frontend et le backend.

Le frontend est accessible via **app.localhost**, tandis que l'API est disponible via **api.localhost**.

Cette configuration simplifie l'accès aux services et limite l'exposition directe des conteneurs.

Dans l'environnement actuel, les communications utilisent **HTTP**. En production, l'ajout d'un certificat **TLS** permettrait de sécuriser les échanges en **HTTPS**.

9.5 Healthchecks et fiabilité des services

Des **healthchecks Docker** sont utilisés afin de vérifier automatiquement la disponibilité des principaux services.

PostgreSQL est contrôlé avec **pg_isready**, tandis que le backend expose une route **/health** permettant de vérifier son bon fonctionnement.

Ces mécanismes ont facilité le diagnostic de plusieurs erreurs de configuration durant le développement et contribuent à améliorer la fiabilité de l'environnement d'exécution.

9.6 Procédure de déploiement de l'environnement

Le déploiement actuel de TaskFlow correspond à la création de l'environnement conteneurisé avec Docker Compose.

Avant le lancement de l'application, Docker et Docker Compose doivent être disponibles sur la machine cible.

Le dépôt du projet doit ensuite être récupéré et les variables d'environnement nécessaires doivent être configurées.

La configuration Docker peut être vérifiée avec la commande :

```
docker compose --env-file .env.docker config
```

Cette commande permet de détecter certaines erreurs de syntaxe ou de structure du fichier Docker Compose.

Les images et les conteneurs peuvent ensuite être construits et lancés avec :

```
docker compose --env-file .env.docker up -d --build
```

Docker Compose crée les services et le réseau nécessaires à l'application.

PostgreSQL démarre et son healthcheck vérifie sa disponibilité.

Lorsque la base est considérée comme saine, le backend peut démarrer.

Le frontend et Traefik sont également lancés dans leurs conteneurs respectifs.

L'état des services peut être contrôlé avec :

```
docker ps
```

Les journaux d'un service peuvent être consultés avec :

```
docker logs <nom-du-conteneur>
```

Le fonctionnement du backend peut être vérifié avec :

<http://api.localhost/health>

L'application frontend est accessible avec :

<http://app.localhost>

Cette procédure permet de recréer l'environnement TaskFlow de manière reproductible à partir du projet et de sa configuration Docker.

9.7 Qualité et maintenabilité du code

Plusieurs bonnes pratiques ont été mises en œuvre afin de garantir la qualité et la maintenabilité de TaskFlow.

Le backend est organisé selon une architecture en couches (**routes**, **controllers**, **services**, **middlewares** et **Prisma**), tandis que le frontend repose sur des **composants React** et des **services** dédiés aux appels API. Cette organisation favorise la séparation des responsabilités et facilite les évolutions du projet.

Le code est versionné avec **Git** et **GitHub**, tandis que le suivi fonctionnel est réalisé avec **Trello** à l'aide de **User Stories**, des critères **Definition of Ready (DoR)** et **Definition of Done (DoD)**.

La qualité du backend est renforcée par des **tests unitaires Jest**, qui valident le service d'authentification et limitent les risques de régression.

Enfin, une démarche d'**intégration continue (CI)** est prévue avec **GitHub Actions** afin d'automatiser l'installation des dépendances, l'exécution des tests et la vérification du build. Le déploiement reste actuellement manuel via Docker Compose, mais pourra évoluer vers un pipeline **CI/CD** complet.

10. Éco-conception et numérique responsable

L'éco-conception a été prise en compte lors du développement de TaskFlow à travers plusieurs choix visant à limiter la complexité et les ressources utilisées.

Une **architecture monolithique** a été privilégiée plutôt qu'une architecture microservices, plus complexe et moins adaptée au périmètre du projet.

Le frontend repose sur des **composants React réutilisables**, limitant la duplication du code et facilitant la maintenance. Les appels à l'API sont centralisés dans des services dédiés, améliorant l'organisation et la réutilisation du code.

La base de données PostgreSQL est structurée autour de quatre entités principales (User, Board, Column et Task), ce qui limite la redondance des données. Les suppressions en cascade définies avec Prisma évitent également la conservation de données orphelines.

Enfin, l'environnement Docker est volontairement limité aux services indispensables (Traefik, frontend, backend et PostgreSQL), afin de conserver une infrastructure adaptée aux besoins du projet.

Dans une version future, cette démarche pourrait être complétée par des mesures de performance, une optimisation des ressources frontend, des images Docker plus légères ainsi qu'un chargement progressif des données.

11. Conclusion

La réalisation de TaskFlow m'a permis de concevoir et développer une application web complète en mettant en œuvre les compétences attendues d'un Concepteur Développeur d'Applications.

Ce projet m'a permis d'approfondir mes connaissances en **React, Node.js, Express, PostgreSQL, Prisma ORM, Docker et Traefik**, tout en appliquant les bonnes pratiques liées à la sécurité, aux tests et à l'organisation d'un projet.

Difficultés rencontrées

Les principales difficultés ont concerné la configuration de PostgreSQL avec Prisma ainsi que la mise en place de Docker et Traefik. L'analyse des journaux Docker et des healthchecks m'a permis d'identifier puis de corriger plusieurs erreurs de configuration, notamment lors de la communication entre le backend et la base de données.

Compétences acquises

Ce projet m'a permis de renforcer mes compétences en développement full-stack, en architecture logicielle, en sécurité (JWT, bcrypt, Express Validator), en conteneurisation avec Docker ainsi qu'en tests unitaires avec Jest.

J'ai également amélioré ma gestion de projet grâce à Trello, aux User Stories et aux critères **Definition of Ready (DoR)** et **Definition of Done (DoD)**.

Améliorations possibles

Plusieurs évolutions pourraient être apportées à TaskFlow, notamment la gestion collaborative des tableaux, l'ajout de commentaires et de notifications, le renforcement de la sécurité avec HTTPS et des cookies HttpOnly, l'extension des tests automatisés ainsi que la mise en place d'un pipeline CI/CD complet.

TaskFlow constitue une expérience concrète qui m'a permis de développer des compétences techniques et méthodologiques tout en me préparant aux exigences du métier de Concepteur Développeur d'Applications.